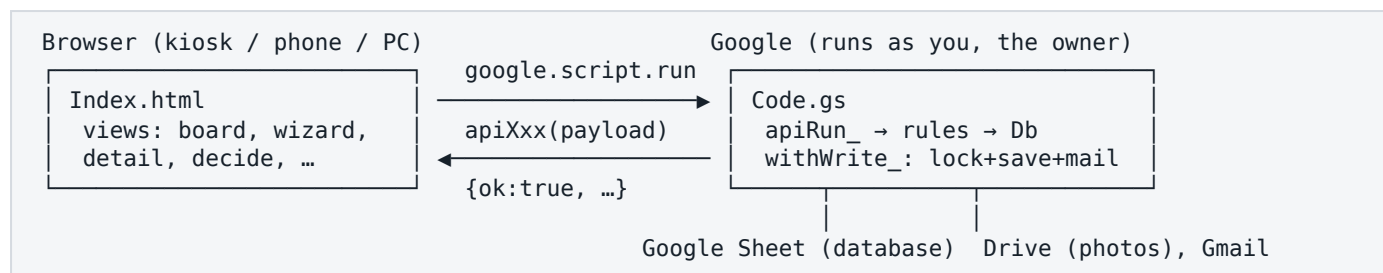


eQRAP Logistika – How the code works

For the two-file version: `Code.gs` (server, runs at Google) and `Index.html` (the page in the browser). Every function named here exists in the code under this name.

1. The big picture



1. The user opens the `/exec` URL. Google runs `doGet(e)`, which builds `Index.html` and puts the user and the settings into it as `window.BOOT`.
2. The page shows a view depending on `?page=` (board, qrap, decide ...). Switching pages does **not** reload – JavaScript redraws the screen.
3. To read or save anything the page calls a server function `apiXxx` through `google.script.run`.
4. Every `apiXxx` returns a plain object with `ok: true/false`. Errors are short codes like `E_REQUIRED`; the page translates them into Czech/English.
5. The server checks every permission itself – a hidden button is not security.

2. Three ideas that the whole server uses

a) `apiRun_` – the standard wrapper

```
function apiSaveAssessment(p) {
  return apiRun_(() => { ...rules...; return { assessment: row }; }, true); // true = it writes
}
```

`apiRun_` checks that the user is signed in, runs the body, and turns any error into `{ok:false, errors:[...]}`. Inside the body, `fail_('E_FORBIDDEN')` stops with one error, `failMany_([...])` with a list (e.g. all missing fields).

b) `withWrite_` – safe saving

For writing calls: take the **script lock** (only one save at a time, so two tablets never get the same number) → read fresh data → run the body → write all changes to the Sheet at once (`dbFlush_`) → write the audit log → send queued e-mails (`mailFlush_`) → clear the board cache. If the body fails, **nothing** is written.

c) The Sheet is read by header name

`SCHEMA` lists every tab and column. `dbTable_('QRAP')` reads a whole tab once and gives rows as objects (`q.status`, `q.what`). Changes are only remembered (`dbInsert_`, `dbUpdate_`) and written together at the end. Nothing is ever deleted – rows get `status REMOVED` or `removed = TRUE`.

3. Code.gs, part by part

`Code.gs` is the old separate files joined; each part starts with a comment banner like `// Config.gs`.

Config – constants, schema, settings

Function / constant	What it does
ST, FINAL_STATES	The QRAP states DRAFT → QR_OPEN → WAIT_DECISION → ... → CLOSED.
SCHEMA	Every tab with its columns and types (text, num, bool, ts = timestamp, date). setup() and the Db code both use it.
getSettings_()	Reads all SET_* tabs into one object (areas, people, shifts, routing, lists). Cached 10 minutes.
cfg_ / cfgNum_ / cfgBool_	One value of SET_CONFIG as text / number / true-false.
route_(area, role)	E-mail addresses from SET_ROUTING for this area and role, for the shift running now (falls back to shift *).
shiftAt_(date)	Which shift is running at a moment (handles the night shift over midnight).
nextShiftStart_(date)	When the next shift starts – deadline for the ④ decision.
onEdit(e)	Runs automatically when you edit the Sheet; editing a SET_* tab clears the settings cache.
clearCaches()	Run by hand if settings look old.

Util – small helpers

Function	What it does
fmt_, todayStr_	Format a date in Prague time.
tzDate_(y,m,d,h,mi)	Builds the exact moment of a Prague wall-clock time (summer/winter time safe).
parseDateTime_	Turns 2026-09-29T10:40 from the browser into a Date (as Prague time).
str_, yesNo_, intOrBlank_	Clean user input: trim and cut text, ANO/NE, whole numbers ≥ 0.
newId_('IA')	Unique id like IA-3F9A1C2B7D.
esc_	Escapes text for HTML e-mails (prevents injected HTML).
clientRow_	Copy of a row that is safe to send to the browser (no Date objects).

Db – the database layer

Function	What it does
dbTable_(name)	One bulk read of a tab (getDataRange().getValues()), columns mapped by header, remembered for the rest of the call.
dbWhere_ / dbFirst_	Filter rows / first matching row.
dbGroup_(tab, 'qrap_id')	Rows grouped by QRAP, e.g. all actions of each QRAP – no extra reads.
dbInsert_ / dbUpdate_	Add / change a row in memory; fill created/updated columns; write the audit entry.
dbFlush_	Writes all remembered changes, one setValues per block of rows.
dbFromCell_ / dbToCell_	Convert between Sheet cells and memory. Text starting with = gets an apostrophe so it can never become a formula.
withWrite_(fn)	Lock → body → save → audit → e-mails → clear cache (see section 2).
nextQrapId_(area)	Next number WH1-2026-9-14 from tab COUNTER (sequence restarts every month).
cachePutBig_ / cacheGetBig_	Store big values in CacheService by splitting them (limit 100 KB per value).

Audit – the change log

audit_() collects entries (who, when, QRAP, field, old → new); auditFlush_() appends them to AUDIT_LOG. appendByHeader_() appends to a log tab reading only its header row (fast even when the log is huge). readLogColumn_() reads one column, e.g. the MAIL_LOG keys.

Auth – who may do what

Function	Rule
<code>currentUser_()</code>	E-mail of the signed-in user + name, roles and areas from SET_PEOPLE. <code>isKiosk</code> if role KIOSK.
<code>requireIdentity_()</code>	Empty e-mail → error E_NO_IDENTITY.
<code>isDecider_(u,q)</code>	May sign: SUPERVISOR of the area (APU_MANAGER for APU copies) and never the kiosk .
<code>canEditQr_</code>	Edit ①②③ until ④ is signed (kiosk, operator, TL, supervisor of the area).
<code>canEditCheck_</code>	Quality may also edit the ③ check numbers.
<code>canDecide_ / canShiftInfo_</code>	Sign ④ / tick "shift informed".
<code>canAnalyze_ / canEditAction_ / canNoteAction_</code>	⑤⑥ for assigned pilots and the decider; the owner of one action may close it; the N+1 may add notes.
<code>canEffect_ / canClose_ / canAssess_ / canManager_</code>	Effectiveness, closure, assessment, manager page.
<code>permissionsFor_</code>	All flags together – the page uses them to show buttons.
<code>isLocked_ / addLock_</code>	<code>locked_parts</code> : QR (①–③), DECISION (④), ANALYSIS (⑤⑥).

Main – entry point

`doGet(e)` renders the page; `bootstrap_` collects user + settings; `include` is kept for the multi-file version; `apiRun_`, `fail_`, `failMany_` are explained in section 2.

Qrap – shared rules of one QRAP

Function	What it does
<code>getQrapRow_(id)</code>	Loads a QRAP or stops with E_NOT_FOUND.
<code>photos0k_</code>	Both photos, or a written exception.
<code>section3State_</code>	Is ③ complete? Risk answered; check data if ANO; ≥ 1 action, all done; process restored or not stopped. Returns the missing items.
<code>refreshContainment_</code>	QR_OPEN → WAIT_DECISION when ③ becomes complete (and back if not).
<code>effState_</code>	Current effectiveness round and its 5 slots (round = 1 + rounds that had NOT_EFFECTIVE).
<code>computeChips_</code>	The 7 coloured squares: done / wait / late / todo / not needed.
<code>waitingFor_</code>	"Čeká na" text code for the board.
<code>closureChecklist_</code>	The closure conditions with ✓/X.
<code>apiGetQrap / qrapDetail_</code>	Everything a detail page needs in one call.

Wizard – the kiosk form (steps 1–5)

Function	What it does
<code>apiSaveStep(p)</code>	Called on every <i>Další</i> . mode <code>next</code> = validate fully; <code>later</code> = "Dokončí TL později"; <code>draft</code> = idle timeout (saves a draft without checks).
<code>saveStep1_ / validateStep1_</code>	① fields and rules (SAFETY, CO, KDY not in future, KOLIK, repeat ...). First save creates the number. SAFETY = ano sends E2 at once.
<code>repeatRefValid_</code>	The repeated QRAP exists, same area, within 7 days.

saveStep2_	Photo exception text; checks photos.
saveStep3_ / setAlerts_	② roles; un-ticked roles are marked removed. Kvalita forced when risk = ANO.
saveStep4_ / validateStep4_ / saveIaRows_	③ risk, check numbers, immediate actions; risk ANO → Kvalita + E3.
apiSubmitQrap	"Odeslat": checks everything, DRAFT → QR_OPEN, sends E1.
apiGetWizard / apiGetDrafts / apiSuggestRepeats	Reopen a draft; list of drafts; similar QRAPs from last 7 days.

Board

apiGetBoard returns rows for one area and filter (open / today / safety / all), newest last, 40 at a time (before = load older). boardRowsCached_ keeps the computed rows 60 s in the cache; buildBoardRows_ reads each child tab once for all QRAPs; boardRow_ makes one row; invalidateBoardCache_ runs after every save.

Decision and Escalation – ④

Function	What it does
apiSaveShiftInfo	One "shift informed" tick (who + when).
apiSignDecision / decisionGuards_	Checks: ③ complete; Solved blocked if SAFETY, comment needed; Continue needs a pilot. Warnings (repeat, wrong pieces, shifts not informed) come back as needConfirm and the page asks. Then: Solved → CLOSED_SOLVED + E9; Continue → pilots + ANALYSIS + E5; Escalate → copy + E6. Locks ①–④.
escalateQrap_	Copy at the parent area (APU) with suffix: ...-14-A; copies ①–③ and photo links; links both ways (parent_id, assign_ref).
nextSuffix_ / baseId_	Next free letter A, B, C ...
apiGetEscalations	Lists for the Escalation page.

Analysis and Actions – ⑤⑥

Function	What it does
apiSaveAnalysis	5 Why chains (no gaps, root on a filled step), lesson learned, standard question. Warns under 3 steps.
syncStdAction_	Standard = ANO creates action "Aktualizovat standard"; NE removes it.
recomputeAnalysisStatus_	Moves ANALYSIS → ACTIONS_OPEN (root + action) → VERIFY (all actions closed).
apiAddPilot	Supervisor adds a pilot later (E5 to them).
apiSaveDefAction / validateDefAction_	Create/edit an action: text, root step, pilot, plan ≥ today, done ≤ today. A done date closes it. New action → E5b.
apiRemoveDefAction	Status REMOVED (kept).
apiAddActionNote / apiSetReviewDone	Notes with review dates.
apiGetMyActions	"Moje akce" – mine or my team's (N+1).

Effectiveness and Assessment

apiSaveEffect records a slot: 5 **different** shifts (date + shift), not in the future; EFFECTIVE is final; NOT_EFFECTIVE → back to ANALYSIS, new round, E11. apiSignClosure signs VERIFY → CLOSED only if closureChecklist_ is all ✓ (E9). apiSaveAssessment stores ⑦ (5 × OK/NOK/NA + feedback); all assessments are kept.

Photos

`apiUploadPhoto` saves the resized full picture and thumbnail to `PHOTO_FOLDER / {number}/` and an ATTACHMENT row (older photo of the same kind marked removed). `apiGetPhoto` returns a photo as base64 – only files listed in ATTACHMENT, because the app runs as the owner and must not expose other Drive files. `photoBlobsForMail_` gives thumbnails for e-mails.

Mail – e-mails E1–E11

Function	What it does
<code>queueQrapMail_</code>	Builds subject <code>[QRAP id] need · SAFETY: ... · Opakování: ...</code> and the HTML body, puts it in a queue.
<code>mailE1_ ... mailE11_</code>	One function per rule: who gets it (from <code>route_</code>), the text, the button page, and a key .
<code>mailFlush_</code>	After saving: skips keys already in MAIL_LOG (so "send once" works), checks the quota, attaches photos, sends, logs.

Rule	When → to
E1	Odeslat → supervisor, cc manager
E2	SAFETY = ano at step 1 → EHS + supervisor
E3	Risk on stock = ANO → Quality
E4	No decision by next shift (hourly check) → supervisor
E5 / E5b	Pokračovat → pilots / new action → action pilot
E6	Escalation → APU manager
E7 / E8	Daily: overdue actions per pilot (+N+1) / manager summary
E9	Closed → finder + supervisor
E10	③ still open 24 h after KDY → TL + supervisor
E11	Not effective → pilots + supervisor

Triggers, Manager, Admin, Setup

Function	What it does
<code>installTriggers / removeTriggers</code>	Create / delete the three timed jobs.
<code>checkReminders</code>	Hourly: E4 and E10.
<code>dailyDigest</code>	At DIGEST_TIME: E7 every day, E8 Mon–Fri.
<code>archiveOld</code>	Sunday night: archive QRAPs closed more than ARCHIVE_AFTER_DAYS ago.
<code>apiGetManager / managerSets_ / needsYou_</code>	Tiles, "Potřebuje vás" sorted by urgency, filtered list.
<code>apiSearch</code>	Search number, text, material, HU, supplier.
<code>apiAdminUnlock / apiAdminReopen / apiAdminArchive</code>	Admin only, reason required, written to AUDIT_LOG.
<code>setup / ensureSheet_ / seedSettings_</code>	Create tabs, headers, formats and example settings (safe to run twice).
<code>selfTest</code>	Report ✓/✗: tabs, config, folder, time zones, routing, triggers.
<code>seedDemoData</code>	9 demo QRAPs, no e-mails.

4. Index.html (the browser side)

Index.html contains CSS, the text dictionary and one `<script>` per view.

Part	Key functions
------	---------------

l18n	<code>t('key', params)</code> returns the text in CZ or EN; <code>setLang</code> switches (remembered on the device).
ClientCore	<code>h(tag, attrs, children)</code> builds HTML safely (user text always as text → no XSS). <code>api(fn, ...)</code> calls the server; on lost connection it shows the red bar and retries, the form keeps its data. <code>go(page, params)</code> switches the view. <code>toast</code> , <code>modal</code> , <code>confirmDlg</code> , <code>warnConfirm</code> dialogs. <code>fmtDT</code> , <code>toLocalInput</code> Prague dates. <code>resizeImage</code> shrinks camera photos before upload. <code>qrCode</code> . <code>Idle</code> timer for the kiosk.
ClientForms	Widgets: <code>field</code> , <code>segmented</code> (big buttons), <code>yesNo</code> , <code>multiToggle</code> , <code>selectEl</code> , <code>personSelect</code> , <code>button</code> (disabled while saving), <code>chipsEl</code> , <code>sectionCard</code> , <code>table</code> .
ViewBoard	Board rows, filters, scroll-up loading, 60 s refresh, drafts list.
ViewWizard + Steps	State <code>Wiz.S</code> ; <code>Wiz.steps[1..5]</code> draw each step; <code>Wiz.validate[n]</code> same checks as the server; <code>next()</code> saves then goes on; <code>onIdle()</code> saves a draft.
ViewQrap	Detail of all 7 sections, buttons from <code>can</code> flags, QR code, admin box.
ViewDecide	④ on the phone and the closure page.
ViewAnalysis, ViewActions, ViewEffect, ViewAssess, ViewManager, ViewEscalation	One page each.

5. One QRAP from start to end

1. **Kiosk** opens the board → `apiGetBoard`.
2. Step 1 *Další* → `apiSaveStep` → `saveStep1_` → `nextQrapId_` → new DRAFT row.
3. Photos → `apiUploadPhoto`. Steps 3–4 → `saveStep3_`, `saveStep4_`.
4. *Odeslat* → `apiSubmitQrap` → QR_OPEN (or WAIT_DECISION) → E1.
5. **Supervisor** scans the QR → `apiSaveShiftInfo`, `apiSignDecision` → ANALYSIS, E5.
6. **Pilot** → `apiSaveAnalysis`, `apiSaveDefAction` → ACTIONS_OPEN → VERIFY.
7. **TL on kiosk** → $5 \times$ `apiSaveEffect`.
8. **Supervisor** → `apiSignClosure` → CLOSED, E9. Anyone allowed → `apiSaveAssessment`.
9. In the background: `checkReminders`, `dailyDigest`, `archiveOld`.

6. How to change things

Add a field (e.g. "Číslo objednávky")

1. Code.gs: add `['order_no', 'text']` to `SCHEMA.QRAP.cols`; run `setup()` (adds the column).
2. Code.gs `cleanStep1_`: `order_no: str_(d.order_no, 60)`,
3. Index.html `Wiz.data[1]`: add `'order_no'`; `Wiz.steps[1]`: add a `field(...textInput...)`.
4. Show it in `sec1` (ViewQrap) and add the text `'f.order_no'` in l18n.
5. Deploy a new version.

Add an e-mail rule

Write `mailE12_(q)` like the others (recipients from `route_`, a unique key), call it where the event happens inside a saving function. `mailFlush_` does the rest.

7. Common problems

Problem	Fix
Changes not visible	Deploy → Manage deployments → Edit → New version.

"Neznámý uživatel"	Browser not signed in with a domain account; deployment must be "within domain".
"Missing tab"	Run <code>setup()</code> ; never rename tabs or headers.
No e-mails	Run <code>selfTest()</code> (routing), look at MAIL_LOG and Executions; check APP_URL.
Photos fail	PHOTO_FOLDER_ID missing or wrong.
Authorization asked again	Run any function in the editor, allow, deploy again.